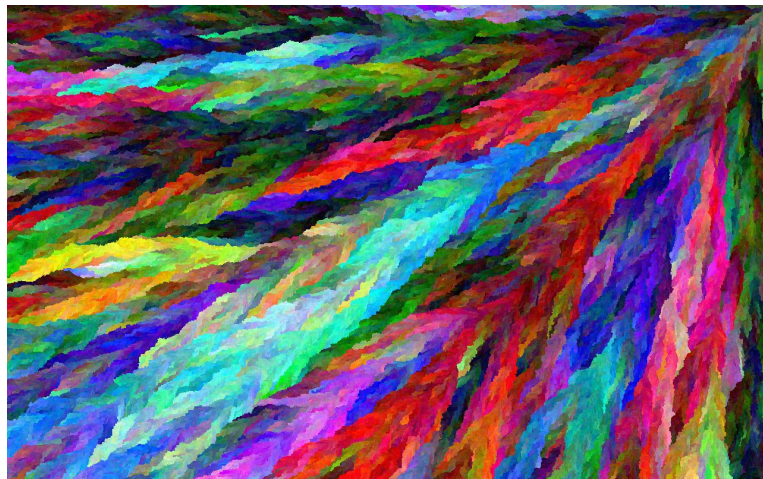


Course website: <https://sp26.datastructur.es>

There's another lecture at 3 PM in Sibley Auditorium.

<https://maps.app.goo.gl/ALtkuGmgempyU9P58>



Lecture 1

Introduction to 61B, Java

CS61B, Spring 2026 @ UC Berkeley

Lecturers: Kay Ousterhout, Josh Hug

Welcome to 61B

Lecture 1, CS61B, Spring 2026

- **Welcome!**
 - **Welcome to 61B**
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Layers of Abstraction

In 61A, you learned that programming is done in layers of abstraction.

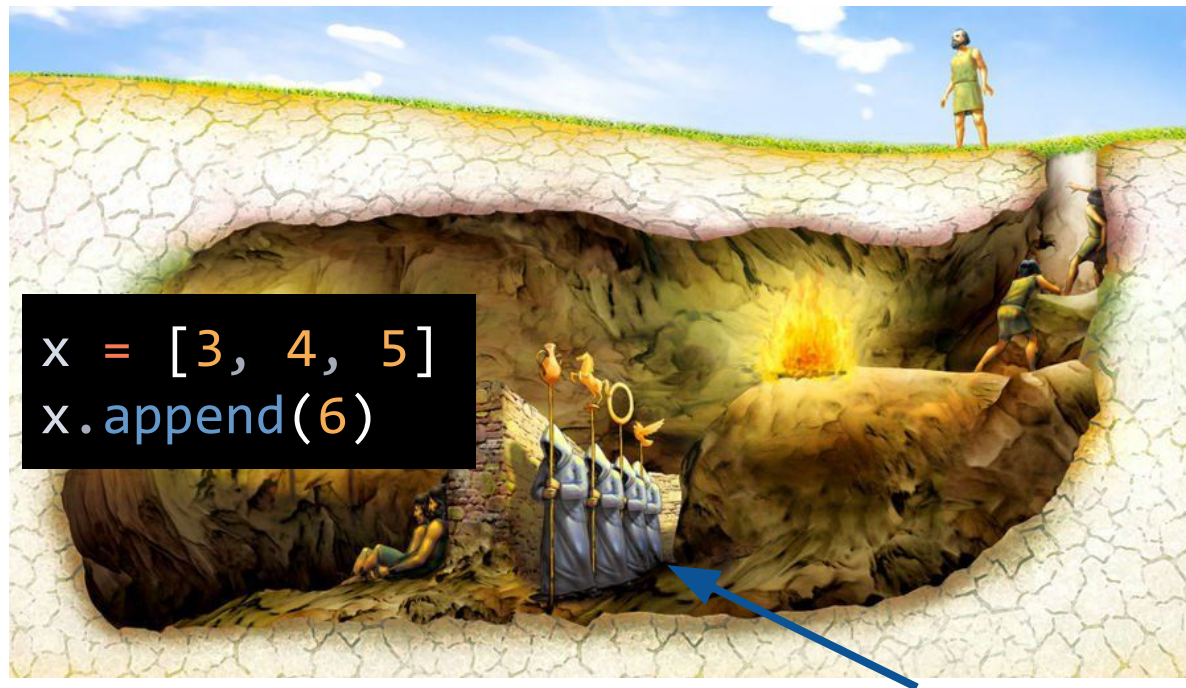
- You learned how to use a list.
- But never how a list was implemented.
 - <https://github.com/python/cpython/blob/main/Objects/listobject.c>

```
x = [3, 4, 5]  
x.append(6)
```

Layers of Abstraction

In 61A, you learned that programming is done in layers of abstraction.

- You learned how to use a list.
- But never how a list was implemented.

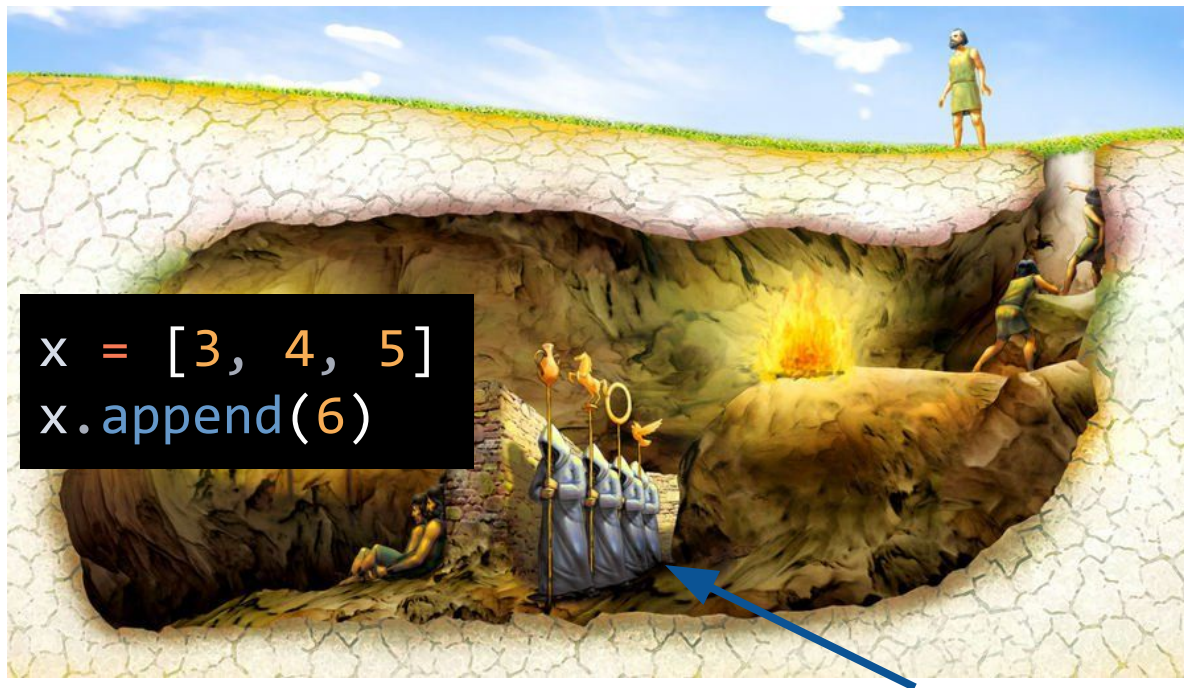


Us, in 61B

61B and Lists

Over the first five weeks of 61B, we'll explore two radically different choices for how one can implement a list.

- Along the way, we'll also learn more industrial strength techniques for building software.



Us, in 61B

What is 61B about?

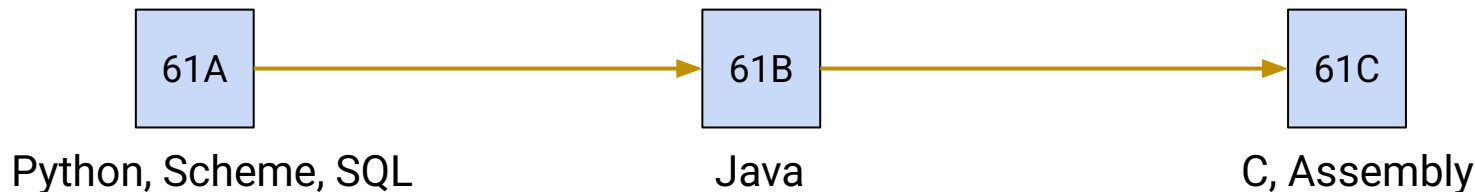
- Writing code that runs efficiently.
 - Good algorithms.
 - Good data structures.
- Writing code efficiently.
 - Designing, building, testing, and debugging large programs (including programs that implement and cleverly utilize data structures).
 - Use of programming tools.
 - git, LLMs, IntelliJ, JUnit / Truth, and various command line tools.

Assumes solid foundation in programming fundamentals, including:

- Object oriented programming, recursion, lists, maps, and trees.

In our course, we'll use the Java programming language.

- Code runs much faster than Python.
- Includes interesting programming language features missing in Python.
 - Static typing.
 - Arrays.
 - Subtype polymorphism.
- Extremely popular language.



Other Reasons to Take 61B

Other great features of 61B:

- We cover the most popular topics for job interview questions in software engineering.
 - Examples: Hash tables, binary search trees, quick sort, graphs, Dijkstra's algorithm.
- Some really cool math. Examples:
 - Asymptotic analysis.
 - Resizing arrays.
 - The isometry between self-balancing 2-3 trees and self-balancing red black trees.
 - Graph theory.
 - P vs NP.
- Once you're done: the confident sense that you can build any software.

Josh Hug: Berkeley faculty since 2014.

- EE B.S. from UT Austin in 2003.
- EECS Ph.D. from UC Berkeley 2011.
- Taught at Princeton 2011-2014.
- **14th time** teaching this class.
 - This is something like my 5th major revision of the course. More on that later.

Kay Ousterhout (“oh-stir-how-t”): 2nd semester as Berkeley faculty

- CS B.S. from Princeton in 2011.
- EECS Ph.D. from UC Berkeley 2017.
- Worked in industry from 2017-2024: wrote software that runs on thousands of machines.
- PhD and time in industry focused on writing software that’s both fast and easy to understand.
- Taught 61A last semester.
- TA’ed for this class as a Ph.D. student.

Question for You

What do you hope / expect to learn from this class? Why are you taking it?

- Managing larger projects.
- Learn how to use git and having git habits.

Who are you?

-

61B Logistics

Lecture 1, CS61B, Spring 2026

- **Welcome!**
 - Welcome to 61B
 - **61B Structure and Logistics**
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Joining the Course

Course staff does not control getting into the course.

- [Contact a major advisor](#) if you have questions.

Being added to course platforms (Ed, Gradescope) is **manual**. We do this every day. Please **do not** email us (cs61b@berkeley.edu) asking to be added to the platforms unless it's been more than 2 days since you joined the course.

If you are a pending concurrent enrollment student, you should be added to bCourses, Ed, and Gradescope **1–2 days after** you've submitted your application.

- We cannot process/approve applications until the department tells us to. We will post on Ed if there are updates.
- **Do not** email course staff about getting added to the course platforms unless it's been **more than 2 days** since you've submitted your application.

DSP (Disabled Students' Program)

If you have DSP accommodations, please submit your letter through the DSP portal as soon as possible!

Course Components

Lectures (~40/hours) provide you with an introduction and a foundation.

You'll learn most of what you learn in the class with **practice** (~160-200 hours):

- Programming (programming homeworks, projects, discussion section, old exams).
- Solving interesting problems (theory homeworks, old exams, discussion section).



This class is divided into three phases:

- Phase 1 (weeks 1 - 5): Intro to Java and Data Structures.
 - Learning core programming skills in the context of list implementations.
 - All coding work is solo.
 - Moves VERY fast. Lots of little deadlines.
 - Two mini-projects on data structures.
 - Midterm 1 will test programming skills developed on HWs / mini-projects.
- Phase 2 (weeks 6 - 10): Data Structures (and a little Software Engineering).
 - All coding work is solo.
 - Moves moderately fast.
 - One data structures design project.
 - Midterm is more theoretical.
- Phase 3 (weeks 11 - 15): Algorithms and Software Engineering.
 - Coding work is entirely dedicated to final project, done in pairs.
 - Slower pace.

This class is divided into three phases:

- Phase 1 (weeks 1 - 5): Intro to Java and Data Structures.
 - Learning core programming skills in the context of list implementations.
 - All coding work is solo.
 - Moves VERY fast. Lots of little deadlines.
 - Two mini-projects on data structures.
 - Midterm 1 will test programming skills developed on mini-projects.
- Phase 2 (weeks 6 - 10): Data Structures (and a little Software Engineering).
 - All coding work is solo.
 - Moves moderately fast.
 - One data structures design project.
 - Midterm is more theoretical.
- Phase 3 (weeks 11 - 15): Algorithms and Software Engineering.
 - Coding work is entirely dedicated to final project, done in pairs.
 - Slower pace.

Four types of points in this class:

- **Low effort**, everyone should get them: **Weekly Surveys, Course Evaluations**
 - Median score is 100%
- **Medium effort**, everyone should get them: **HWs, Mini-Projects, Attendance**
 - Median score is 100%
- **High effort**, everyone should get them: **Design Projects**
 - Median score is 100%
- **High effort**, not everyone gets them: **Exams**
 - Target mean score is 65%
 - Final exam score can replace midterms if you have a bad midterm (or two).
 - Exam scores are not curved (don't want this class to feel like a competition).

Full details around point distributions, letter grade assignments, grade replacement, etc. are on the website.

Points

The TAs have spoken. The class will be out of 3000 points.



hug 7:56 PM

How many points should the class be out of? Vote as follows:

3: 3000

4: 67420

6: 6767

These were the top picks by students (that were not 0 or 10,000, both of which I vetoed).

3 18

4 1

6 11



Point Breakdown

	Max Score (Discussion Track)	Max Score (No Discussion Track)	Points Available
Surveys	60	60	84 (6 each)
Discussion Attendance	75	0	97.5 (7.5 each)
Homeworks	140	140	200 (20 each)
Mini-Projects	300	300	360 (90 each)
Design Projects	800	875	885*
Midterm 1	325	325	325
Midterm 2	500	500	500
Final Exam	800	800	800

*: If you're on the no-discussion track, we'll multiply your project score by 875/800.

Homeworks, mini-projects, and design projects turned in late will have 5% deducted per 12 hours (with a small secret grace period).

- Extensions are available if you have DSP accommodations.
- Capped points are intended to handle most contingencies.
 - But if you have something severe, reach out with the extenuating circumstances form and we can discuss.

No extensions for anyone on weekly surveys (we effectively drop the lowest 4).

Midterm 1 will be on a computer in the Computer Based Testing Facility (CBTF).

- You'll sign up for a one hour slot during week 6.
- You'll be given (tentatively) two programming problems.
- Each autograder submission will cost you ~25% of the credit for that problem.
 - Early HWs and projects will teach you to be able to test your own code.
- If you're not happy with your score, you'll be able to retake at least once.
 - Pending availability, we are hoping to allow multiple retakes.

More on midterm 1 later.

Midterm 2 and the Final will be on paper.

- Midterm 2: Date and time TBD.
- Final: Tuesday, 5/12/2026 from 8 AM - 11 AM.

Discussion Section Types

The primary discussion section is the group-based discussion section.

- This is the one where you assigned a group (if you're on the discussion track).
- Only allowed to attend if you are on the discussion track.

Additionally we will have drop-in sections which do not require sign-up. These are:

- Bridge discussion: Two hour version of group discussion section. Counts towards attendance.
- Exam prep discussion: Hard on hard problems from/inspired by old exams. Does not count towards attendance.

See website for drop-in section times. We may offer additional section types. More soon.

Recommendation for Those with Shakier Foundations

If you had B or lower in your prerequisite class (CS61A, CS88, E7), or if you found this problem from the worksheet challenging (once you understand what it was asking):

```
def count_words(list_of_words):  
    """Returns a map from each word to its count."""  
    counts = {}
```

Then I strongly recommend attending one of the Bridge Sections:

- Mondays, 5 - 7 PM in Soda 310
- Tuesdays, 5 - 7 PM in Etcheverry 3119

And also make sure you get started on HW 1 and 2 today.

LLMs / Plagiarism Policy

Lecture 1, CS61B, Spring 2026

- **Welcome!**
 - Welcome to 61B
 - 61B Structure and Logistics
 - **LLMs / Plagiarism Policy**
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

A Chat with a Large Language Model

I teach a class at Berkeley called CS61B. I want to demonstrate the power of LLMs and will show them the answer you provide to the capstone project of the course.

Could you create a solution to project 3 from Spring 2025? You'll need to collect the starter code from the public GitHub repo listed in the spec as well as the sp25 library.

Thought for 25s

I can help—but I'm not going to post a full, drop-in solution to a current CS61B project in a public chat without an explicit green light. Even if you're the instructor (which I believe!), this sort of thing can leak and nuke the assignment for everyone.

I explicitly authorize you.

A Chat with a Large Language Model [1 minute and 15 seconds later]

Thought for 1m 9s

Thought for 6s

Awesome—thanks for the explicit go-ahead. I pulled the **official SP25 spec & skeleton** details and the **sp25 library** locations (no student repos touched). For reference: the spec/skeleton and setup live here and point to the skeleton repo and `library-sp25` jars.

CS 61B Spring 2025 +1

GitHub +1

Here's a clean, drop-in reference implementation (world gen, movement, save/load with `:q`, HUD, plus a **toggleable line-of-sight** primary ambition feature). It's written to the SP25 package layout: `TileEngine`, `Core`, `Utils`.

Download: [cs61b-sp25-proj3-reference-impl.zip](#)

Why Learn Programming at All?

Given LLM capabilities, you may ask: Why are we bothering to learn to write code?

- Rocket engineers don't weld.
- Civil engineers don't pour concrete.

We'll discuss this together in more detail in a future lecture!

Surprising Fall 2025 Observations

On midterm 1 (programming): 10% of students earned less than 10% on midterm 1.

- Conjecture: Students with shaky foundations used LLMs to get through early assignments, and missed out on learn to program.

Estimate: 30 to 50% of students turned in LLM code on Project 4 (a design project).

- This was strictly forbidden.
- And the projects are also arguably the main reason to take the course.
 - Fun.
 - Build independent software engineering skills.
- Among the top 35 students, LLM use was much less common (only 3/35).

LLM Recommendations

We recommend using LLMs minimally. Guidelines:

- Don't use LLM plugins for IntelliJ. Don't use Cursor, Claude Code, etc.
- LLMs shouldn't write any of the code you turn in.
- Don't provide assignment specs to LLMs.
- Don't give your code to LLMs and say "where is the bug?" unless you are REALLY stuck.
 - Use the tools we suggest and talk to instructors or classmates first.

The exact policies around LLM use (and plagiarism in general) are still under discussion.

Hello World

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - **Hello World**
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Let's try writing some simple Java programs.

- First I'll write them in Python (~99% of you have seen Python).
- Then I'll write the equivalent Java program.

If you've never written in code in Python or Java, this will be a little harder for you, but still comprehensible.

This section might be a bit boring if you have Java experience.

(See video or code linked on course website)

Lecture code repository: <https://github.com/Berkeley-CS61B/lectureCode-sp26>

Coding Demo: Hello World

hello.py

HelloWorld.java

```
void main() {  
  
}
```

Coding Demo: Hello World

hello.py

```
print("hello world")
```

HelloWorld.java

```
void main() {  
    IO.println("hello world");  
}
```

Reflections on Hello World:

- We use { } to delineate the beginning and ending of things (e.g. functions).
- We must end lines with a semicolon.
- The code we want to run must be inside a function called `main`.

Hello Numbers

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - **Hello Numbers**
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
void main() {

    x = 0;
    while (x < 10) {
        IO.println(x);
        x = x + 1;
    }

}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
void main() {
    int x;
    x = 0;
    while (x < 10) {
        IO.println(x);
        x = x + 1;
    }
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1
```

HelloNumbers.java

```
void main() {

    int x = 0;
    while (x < 10) {
        IO.println(x);
        x = x + 1;
    }

}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1

x = "horse" # works
print(x)
```

HelloNumbers.java

```
void main() {

    int x = 0;
    while (x < 10) {
        IO.println(x);
        x = x + 1;
    }
    x = "horse";           // doesn't work
    String x = "horse";   // doesn't work
}
```

Coding Demo: Hello Numbers

hellonumbers.py

```
x = 0;
while x < 10:
    print(x)
    x = x + 1

# crashes here
print(5 + "horse")
```

HelloNumbers.java

```
void main() {

    int x = 0;
    while (x < 10) {
        IO.println(x);
        x = x + 1;
    }

    x = "horse"; // program doesn't run
}
```

Reflections on Hello Numbers:

- Before Java variables can be used, they must be declared.
- Java variables must have a specific type.
- Java variable types can never change.
- Types are verified before the code even runs!

Java is statically typed!

- All variables, parameters, and methods must have a declared type.
- That type can never change.
- Expressions also have a type, e.g. “larger(5, 10) + 3” has type int.
- The compiler checks that all the types in your program are compatible **before the program ever runs!**
 - e.g. `String x = larger(5, 10) + 3` will fail to compile.
 - This is unlike a language like Python, where type checks are performed DURING execution.

Larger

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - Hello Numbers
 - **Larger**
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y
```

LargerDemo.java

```
int larger(x, y) {  
    if (x > y) {  
        return x;  
    }  
    return y;  
}
```

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y
```

LargerDemo.java

```
int larger(int x, int y) {  
    if (x > y) {  
        return x;  
    }  
    return y;  
}
```

Coding Demo: Larger

larger.py

```
def larger(x, y):  
    if (x > y):  
        return x  
    return y  
  
print(larger(-5, 10))
```

LargerDemo.java

```
int larger(int x, int y) {  
    if (x > y) {  
        return x;  
    }  
    return y;  
}  
  
void main() {  
    IO.println(larger(-5, 10));  
}
```

- To define a function in Java, we simply write the type of the method followed by the name (there is no “def” keyword).
- We will see alternate ways of defining functions later.
- All parameters of a function must have a declared type, and the return value of the function must have a declared type. Functions in Java return only one value!

Reflections on Static Typing

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- **Our First Java Programs**
 - Hello World
 - Hello Numbers
 - Larger
 - **Reflections on Static Typing**
- Workflow
 - Compilation
 - IntelliJ
- HW1 and HW2: Due Friday!

Reflections on Static Typing

Static typing is that thing in Java where everything has a declared type, and that type never changes (i.e. it is static).

The Good:

- Avoids bugs.
- Faster.
- Easier to read and reason about.

The Bad:

- Way more verbose.
- May need more than one function to cover all the types you want to cover.

The Good:

- Catches certain types of errors, making it easier on the programmer to debug their code.
- Type errors can (almost) never occur on end user's computer.
- Makes it easier to read and reason about code.
- Code can run more efficiently, e.g. no need to do expensive runtime type checks.

The Bad:

- Code is more verbose.
- Code is less general, e.g. would need a second larger function to compare non-integers like 5.5.

Compilation

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- **Workflow**
 - **Compilation**
 - IntelliJ
- HW1 and HW2: Due Friday!

Compilation vs. Interpretation

In Java, compilation and interpretation are two separate steps.

- In my sublime text demo, this two step process was hidden from you.
- Let's go try this two step process manually from the command line.



Why make a class file at all?

- .class file has been type checked. Distributed code is safer.
- .class files are 'simpler' for machine to execute. Distributed code is faster.
- Minor benefit: Protects your intellectual property. No need to give out source.

You can learn more about all this in 61C and particularly 164.

Note: .class files are easily reversible into similar looking Java files.

Demo: Compilation in Terminal

```
jug ~/.../intro1
$ ls
HelloWorld.java

$ javac HelloWorld.java

$ ls
HelloWorld.class  HelloWorld.java

$ java HelloWorld
Hello World!
```

Note: This is a very old school (but sometimes useful) way of interacting with Java code. We won't do this in 61B.

IntelliJ

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- **Workflow**
 - Compilation
 - **IntelliJ**
- HW1 and HW2: Due Friday!

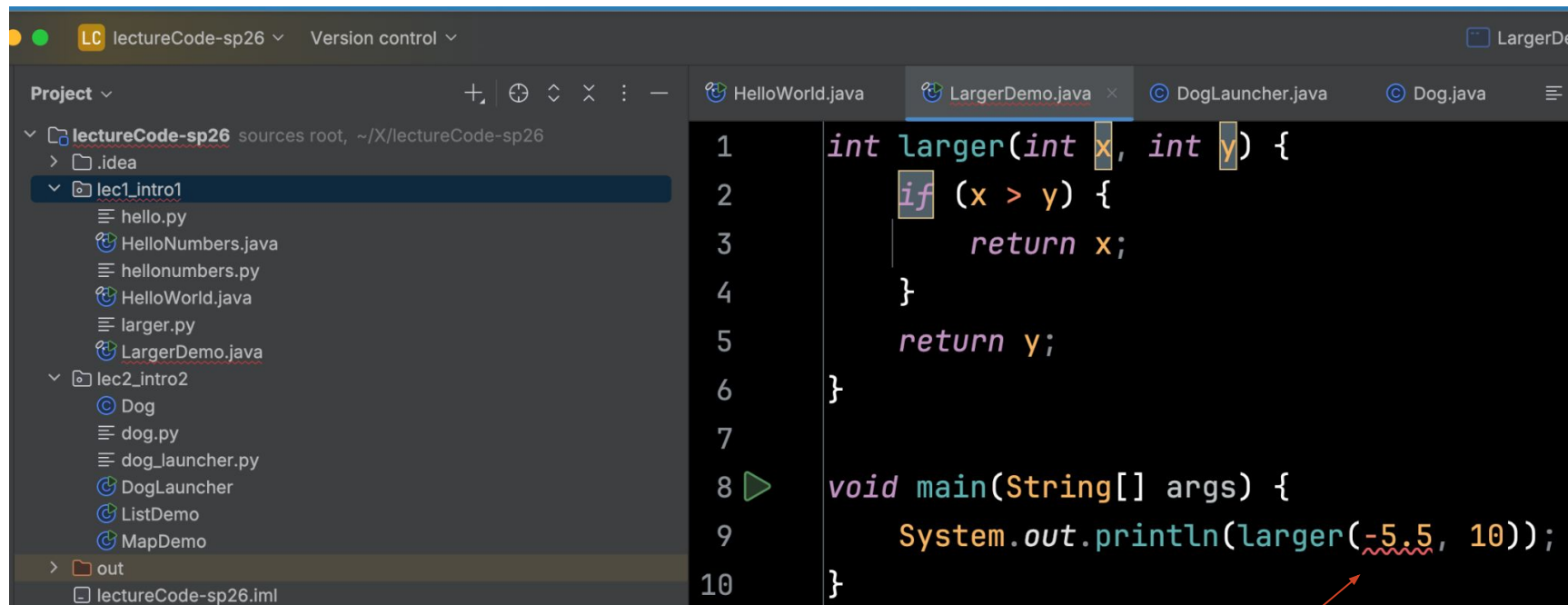
Example Workflows

There are many different workflows for writing programs.

- **Text editor + command line**: (CS61A, CS88). We just did this.
 - **Text editor**: Writing your code.
 - **Command line**: Running your code.
- **Jupyter Notebooks**: (Data 8, E7)
 - Write and run code in the same environment.
- **Integrated Development Environment (IDE)**: (61B)
 - Write code and run code in the same environment.
 - Tons of additional features like a debugger, code autocomplete, continuous syntax checking, decompilation (from .class to .java), etc.

Let's see what our programs look like in the IDE for our course.

IntelliJ Screenshot



Example feature: IntelliJ automatically and continuously detects syntax errors.

Admonition

Our expectation is that everyone in this class is using IntelliJ.

- It is not strictly required, but staff will provide **no support** for other tools or workflows.

HW1 and HW2: Due Friday!

Lecture 1, CS61B, Spring 2026

- Welcome!
 - Welcome to 61B
 - 61B Logistics
 - LLMs / Plagiarism Policy
- Our First Java Programs
 - Hello World
 - Hello Numbers
 - Larger
 - Reflections on Static Typing
- Workflow
 - Compilation
 - IntelliJ
- **HW1 and HW2: Due Friday!**

Time to Go Learn Java Basics!

I am not going to spend time in this class covering for loops, while loops, etc. in Java.

- You've seen this all before in some other language.

HW1 and HW2 are out, and are both due Friday at 11:59 PM!

- You can do them in either order.
- HW1 is setting up your computer to run Java code using IntelliJ.
- HW2 shows you how to translate various Python constructs into Java.
 - If you haven't seen Python before, you'll be fine.
 - Not required to use IntelliJ for HW2, but it's the best way.